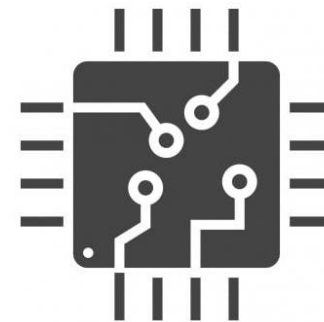




Unidade Acadêmica de
Belo Jardim

Microcontroladores e Microprocessadores



&

Engenharia de Sistemas Embarcados

Engenharia de Controle e Automação



Prof. Henrique Patriota

UABJ

Eng. de Controle
e Automação

PROGRAMAÇÃO EM C

TIPOS DE VARIÁVEIS

DIRETIVAS

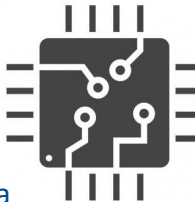
TÉCNICAS DE BITWISE

Shift Left

DICAS DE LEITURA: Cap. 3, 4 e 6 do Almeida, “Programação de Sistemas Embarcados”
OU Cap. 4 do Villaça, “AVR e Arduino”

Microcontroladores e
Microprocessadores

Semana 2 – Aula 2



UABJ

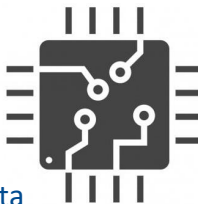
Eng. de Controle
e Automação

COMPILAÇÃO

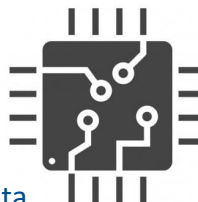
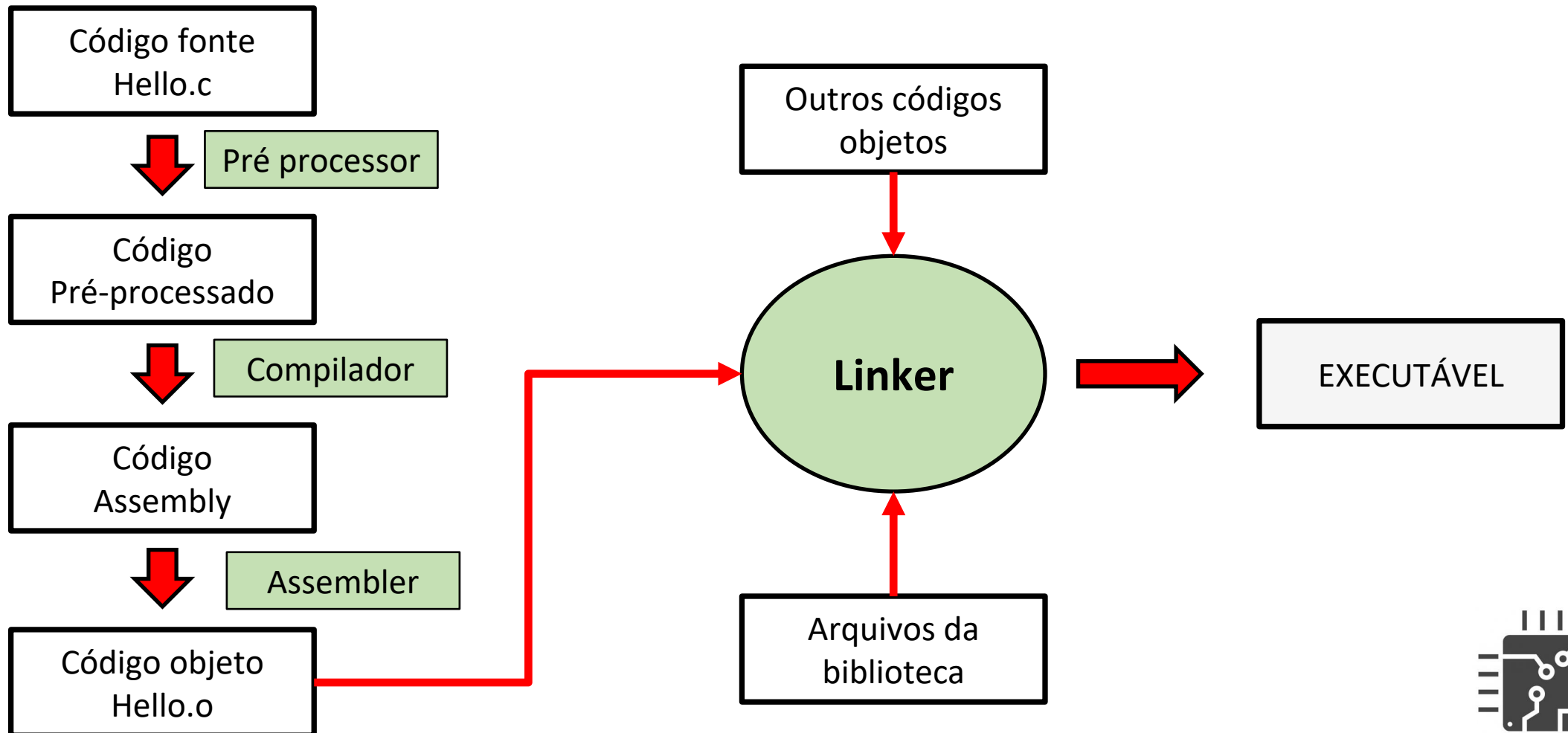
- Pré processamento
- Compilador
- Assemblador
- Linker

Microcontroladores e
Microprocessadores

Semana 2 – Aula 2



Etapas da compilação



O pré processador

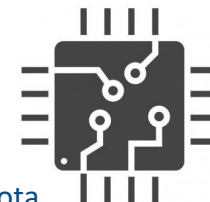
Uma ferramenta de substituição de textos

#define – Substitui uma macro

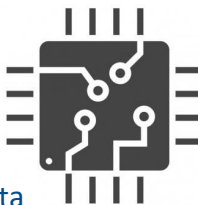
#include – Inclui um texto de outro arquivo

#if – Testa uma condição para compilar

Outros...



O domínio da linguagem *assembly* era total nos primórdios da programação de microcontroladores. Como o *assembly* é uma linguagem com baixo nível de abstração, o tempo gasto e o esforço mental são consideráveis no desenvolvimento de programas. Com a evolução tecnológica, o *assembly* foi quase que totalmente substituído pela linguagem C. Desta forma, os compiladores tem um papel importante, devendo eficientemente converter os programas em C para a linguagem *assembly* (compilação) e, posteriormente, para a de máquina (montagem).



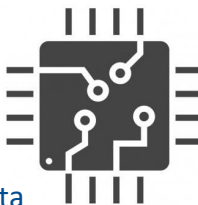
UABJ

Eng. de Controle
e Automação

Portabilidade. Transferir o código de um tipo de microcontrolador para outro, mesmo com arquiteturas totalmente diferentes, é relativamente simples, fato complicado de ser realizado em *assembly*. A portabilidade é uma característica importantíssima, pois permite a migração rápida de uma tecnologia para outra.

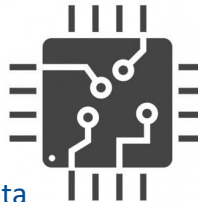
Microcontroladores e
Microprocessadores

Semana 2 – Aula 2



INDENTAÇÃO

Código indentado	Código não indentado
<pre>void main(void) { unsigned int i; unsigned int temp; unsigned int teclanova=0; InicializaSerial(); InicializaDisplays(); InicializaLCD(); InicializaAD(); for (;;) { AtualizaDisplay(); if (teclanova != Tecla) { teclanova = Tecla; for(i=0;i<16;i++) { if (BitTst(Tecla,i)) { EnviaDados(i+48); } } } } for(i = 0; i < 1000; i++); }</pre>	<pre>void main(void) { unsigned int i; unsigned int temp; unsigned int teclanova=0; InicializaSerial(); InicializaDisplays(); InicializaLCD(); InicializaAD(); for (;;) { AtualizaDisplay(); if (teclanova != Tecla) { teclanova = Tecla; for(i=0;i<16;i++) { if (BitTst(Tecla,i)) { EnviaDados(i+48); } } } } for(i = 0; i < 1000; i++); }</pre>



FUNÇÃO PRINCIPAL – MAIN

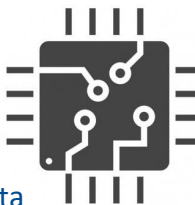
Todo sistema precisa ser iniciado de algum lugar. Os microcontroladores, assim que ligados, começam a executar os códigos a partir de um endereço pré-definido.

Para vários processadores, é necessário fazer um conjunto de configurações iniciais para que seja possível executar o programa em linguagem C.

Entre essas configurações temos a alocação do ponteiro de stack, configuração da página de memória ativa ou, até mesmo, o carregamento do código do programa para a memória RAM. Este procedimento é executado logo após o sistema ser ligado. O código para essa inicialização é feito geralmente em assembly e implementado pelo próprio compilador. Depois de sua execução essa rotina inicial é chamada a função **main()**.

A função **main()** é, portanto, a primeira função que pode executar algum código planejado pelo programador.

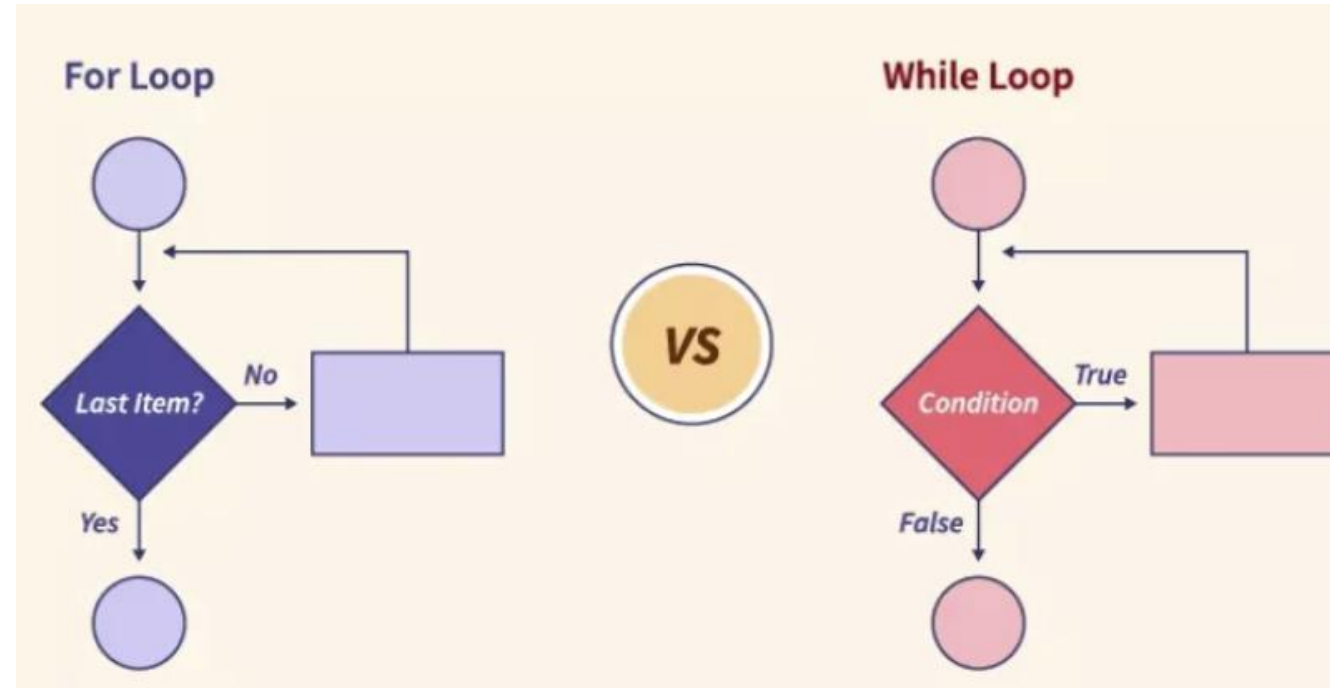
```
1 void main(void) {  
2     //aqui entra o código do programa  
3 }
```



UABJ

Eng. de Controle
e Automação

LOOP INFINITO

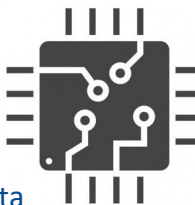


```
for(;;){
```

```
}
```

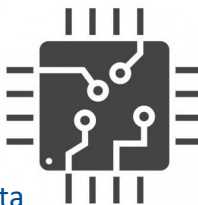
```
while(1){
```

```
}
```



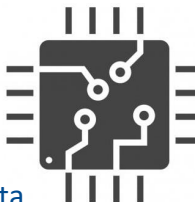
TIPOS DE VARIÁVEIS

<i>Tipo</i>	<i>Bits</i>	<i>Bytes</i>	<i>Faixa de valores</i>
char	8	1	-127 à 127
int	16	2	-32.768 à 32.767
float	32	4	$3,4 \times 10^{-38}$ à $3,4 \times 10^{38}$
double	64	8	$3,4 \times 10^{-308}$ à $3,4 \times 10^{308}$



MODIFICADORES

<i>Tipo</i>	<i>Bytes</i>	<i>Excursão máxima</i>
unsigned char	1	0 à 255
signed char	1	-128 à 127
unsigned int	2	0 à 65.535
signed int	2	-32.768 à 32.767
long int	4	-2.147.483.648 à 2.147.483.647
unsigned long int	4	0 à 4.294.967.295
short int	2	-32.768 à 32.767



DIRETIVAS

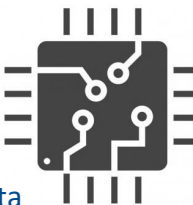
`#include`

A diretiva de compilação `#include` é a responsável por permitir que o programador utilize no seu código funções que foram implementadas em outros arquivos, seja por ele próprio ou por outras pessoas. Não é necessário possuir o código fonte das funções que se deseja utilizar. É necessário apenas de um arquivo que indique os protótipos das funções (como elas devem ser chamadas) e possuir a função disponível em sua forma compilada.

Em geral um arquivo que possui apenas protótipos de funções é denominado de “Header” e possui a extensão “.h”.

`#define`

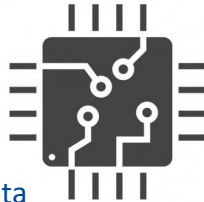
Outra diretiva muito conhecida é a `#define`. Geralmente é utilizada para definir uma constante, mas pode ser utilizada para que o código fonte seja modificado antes de ser compilado.



DIRETIVAS

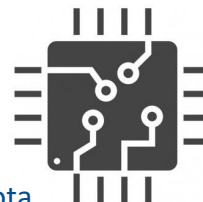
Original	Compilado	Resultado na Tela
<pre>#define CONST 15 void main(void) { printf("%d", CONST * 3); }</pre>	<pre>void main(void) { printf("%d", 15 * 3); }</pre>	45

Função Original	Opções de uso com o #define	Resultado na Tela
<pre>void MostraSaidaPadrao() { #ifdef PADRAO Serial char * msg = "SERIAL"; #else char * msg = "LCD"; #endif printf(msg); }</pre>	<pre>#include <stdio.h> #define PADRAO Serial void main(void) { MostraSaidaPadrao(); }</pre>	SERIAL
	<pre>#include <stdio.h> #define PADRAO LCD void main(void) { MostraSaidaPadrao(); }</pre>	LCD



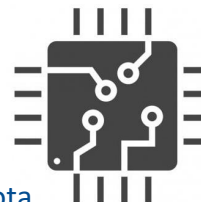
TÉCNICAS DE BITWISE – SHIFT LEFT – SET BIT

Operação	Bit set
Passo a Passo	<pre>char bit = 2; char mascara; mascara = 1 << bit; arg = arg mascara; //em 1 linha arg = arg (1<<bit); //ou arg = (1<<bit);</pre>
Exemplo de uso	<pre>//Ligando o bit 2 da porta D PORTD = PORTD (1<<2); //ou PORTD = (1<<2);</pre>
Com define	<pre>#define BitSet(arg,bit) ((arg) = (1<<bit))</pre>
Exemplo de uso com define	<pre>//Ligando o bit 2 da porta D BitSet(PORTD,2);</pre>



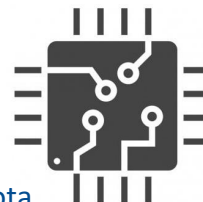
TÉCNICAS DE BITWISE – SHIFT LEFT – CLEAR BIT

Operação	Bit clear
Passo a Passo	<pre>char bit = 2; char mascara; mascara = 1 << bit; arg = arg & ~mascara; //em 1 linha arg = arg & ~(1<<bit); //ou arg &= ~(1<<bit);</pre>
Exemplo de uso	<pre>//Desligando o bit 2 da porta D PORTD = PORTD & ~(1<<2); //ou PORTD &= ~(1<<2);</pre>
Com define	<pre>#define BitClr(arg, bit) ((arg) &= ~(1<<bit))</pre>
Exemplo de uso com define	<pre>//Desligando o bit 2 da porta D BitClr(PORTD, 2);</pre>



TÉCNICAS DE BITWISE – SHIFT LEFT – TOGGLE BIT

Operação	Bit flip
Passo a Passo	<pre>char bit = 2; char mascara; mascara = 1 << bit; arg = arg ^ mascara; //em 1 linha arg = arg ^ (1<<bit); //ou arg ^= (1<<bit);</pre>
Exemplo de uso	<pre>//Trocando o valor do bit 2 da porta D PORTD = PORTD ^ (1<<2); //ou PORTD ^= (1<<2);</pre>
Com define	<pre>#define BitFlp(arg, bit) ((arg) ^= (1<<bit))</pre>
Exemplo de uso com define	<pre>//Trocando o valor do bit 2 da porta D BitFlp(PORTD, 2);</pre>



UABJ

Eng. de Controle
e Automação

Microcontroladores e
Microprocessadores

Semana 2 – Aula 2

FIM

